



```
In [ ]: import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [27]: train = pd.read_csv("train.csv")
features = pd.read_csv("features.csv")
stores = pd.read_csv("stores.csv")
```

```
In [28]: print(train.head())
print(features.head())
print(stores.head())
```

	Store	Dept	Date	Weekly_Sales	IsHoliday
0	1	1	2010-02-05	24924.50	False
1	1	1	2010-02-12	46039.49	True
2	1	1	2010-02-19	41595.55	False
3	1	1	2010-02-26	19403.54	False
4	1	1	2010-03-05	21827.90	False

	Store	Date	Temperature	Fuel_Price	MarkDown1	MarkDown2	\
0	1	2010-02-05	42.31	2.572	NaN	NaN	
1	1	2010-02-12	38.51	2.548	NaN	NaN	
2	1	2010-02-19	39.93	2.514	NaN	NaN	
3	1	2010-02-26	46.63	2.561	NaN	NaN	
4	1	2010-03-05	46.50	2.625	NaN	NaN	

	MarkDown3	MarkDown4	MarkDown5	CPI	Unemployment	IsHoliday
0	NaN	NaN	NaN	211.096358	8.106	False
1	NaN	NaN	NaN	211.242170	8.106	True
2	NaN	NaN	NaN	211.289143	8.106	False
3	NaN	NaN	NaN	211.319643	8.106	False
4	NaN	NaN	NaN	211.350143	8.106	False

	Store	Type	Size
0	1	A	151315
1	2	A	202307
2	3	B	37392
3	4	A	205863
4	5	B	34875

Merge Data

```
In [29]: df = train.merge(features, on=["Store", "Date"], how="left")
df = df.merge(stores, on="Store", how="left")
```

Check Final Dataset

```
In [30]: print(df.head())
print(df.info())
```

	Store	Dept	Date	Weekly_Sales	IsHoliday_x	Temperature	\
0	1	1	2010-02-05	24924.50	False	42.31	
1	1	1	2010-02-12	46039.49	True	38.51	
2	1	1	2010-02-19	41595.55	False	39.93	
3	1	1	2010-02-26	19403.54	False	46.63	
4	1	1	2010-03-05	21827.90	False	46.50	

	Fuel_Price	MarkDown1	MarkDown2	MarkDown3	MarkDown4	MarkDown5	\
0	2.572	NaN	NaN	NaN	NaN	NaN	
1	2.548	NaN	NaN	NaN	NaN	NaN	
2	2.514	NaN	NaN	NaN	NaN	NaN	
3	2.561	NaN	NaN	NaN	NaN	NaN	
4	2.625	NaN	NaN	NaN	NaN	NaN	

	CPI	Unemployment	IsHoliday_y	Type	Size
0	211.096358	8.106	False	A	151315
1	211.242170	8.106	True	A	151315
2	211.289143	8.106	False	A	151315
3	211.319643	8.106	False	A	151315
4	211.350143	8.106	False	A	151315

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 421570 entries, 0 to 421569

Data columns (total 17 columns):

#	Column	Non-Null Count	Dtype
0	Store	421570 non-null	int64
1	Dept	421570 non-null	int64
2	Date	421570 non-null	object
3	Weekly_Sales	421570 non-null	float64
4	IsHoliday_x	421570 non-null	bool
5	Temperature	421570 non-null	float64
6	Fuel_Price	421570 non-null	float64
7	MarkDown1	150681 non-null	float64
8	MarkDown2	111248 non-null	float64
9	MarkDown3	137091 non-null	float64
10	MarkDown4	134967 non-null	float64
11	MarkDown5	151432 non-null	float64
12	CPI	421570 non-null	float64
13	Unemployment	421570 non-null	float64
14	IsHoliday_y	421570 non-null	bool
15	Type	421570 non-null	object
16	Size	421570 non-null	int64

dtypes: bool(2), float64(10), int64(3), object(2)

memory usage: 49.0+ MB

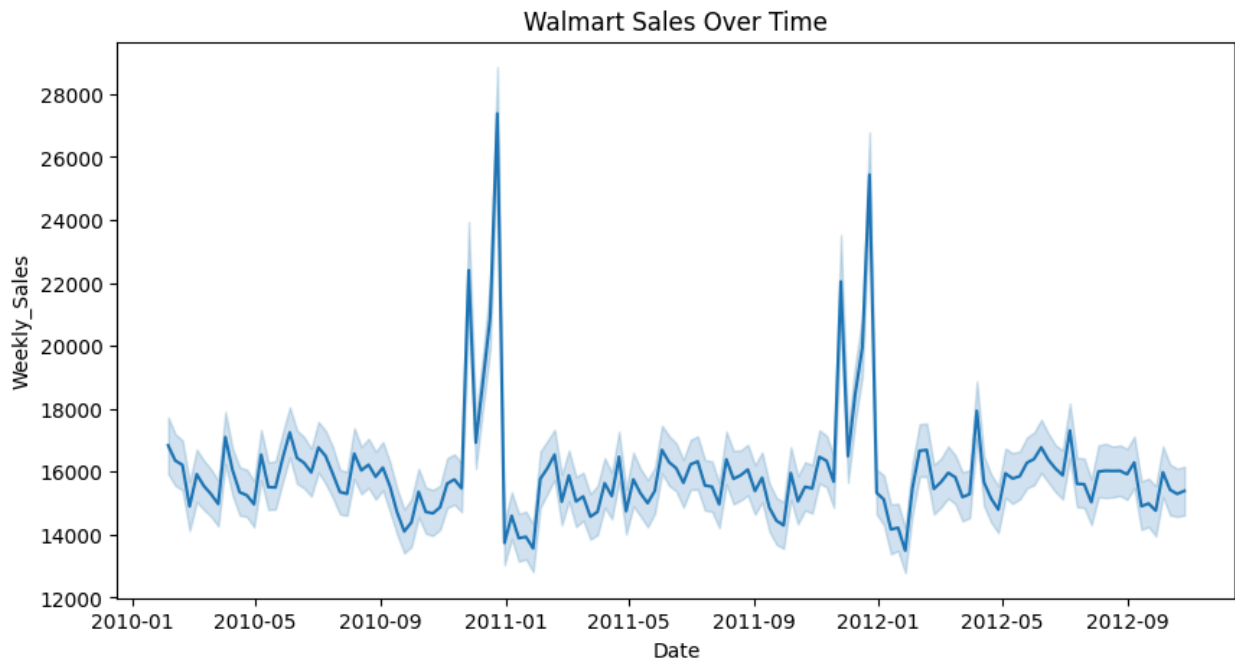
None

Convert Date

```
In [31]: df["Date"] = pd.to_datetime(df["Date"])
```

Basic Visualization

```
In [35]: plt.figure(figsize=(10,5))
sns.lineplot(data=df, x="Date", y="Weekly_Sales")
plt.title("Walmart Sales Over Time")
plt.show()
```



Merge All Data

```
In [36]: df = train.merge(features, on=["Store", "Date"], how="left")
df = df.merge(stores, on="Store", how="left")
```

Check Dataset

```
In [37]: print(df.head())
```

	Store	Dept	Date	Weekly_Sales	IsHoliday_x	Temperature	\
0	1	1	2010-02-05	24924.50	False	42.31	
1	1	1	2010-02-12	46039.49	True	38.51	
2	1	1	2010-02-19	41595.55	False	39.93	
3	1	1	2010-02-26	19403.54	False	46.63	
4	1	1	2010-03-05	21827.90	False	46.50	

	Fuel_Price	Markdown1	Markdown2	Markdown3	Markdown4	Markdown5	\
0	2.572	NaN	NaN	NaN	NaN	NaN	
1	2.548	NaN	NaN	NaN	NaN	NaN	
2	2.514	NaN	NaN	NaN	NaN	NaN	
3	2.561	NaN	NaN	NaN	NaN	NaN	
4	2.625	NaN	NaN	NaN	NaN	NaN	

	CPI	Unemployment	IsHoliday_y	Type	Size
0	211.096358	8.106	False	A	151315
1	211.242170	8.106	True	A	151315
2	211.289143	8.106	False	A	151315
3	211.319643	8.106	False	A	151315
4	211.350143	8.106	False	A	151315

Check Missing Values

```
In [38]: print(df.isnull().sum())
```

```
Store          0
Dept           0
Date           0
Weekly_Sales   0
IsHoliday_x    0
Temperature     0
Fuel_Price     0
Markdown1      270889
Markdown2      310322
Markdown3      284479
Markdown4      286603
Markdown5      270138
CPI            0
Unemployment    0
IsHoliday_y     0
Type           0
Size           0
dtype: int64
```

Handle Missing Values

```
In [39]: df.fillna(df.median(numeric_only=True), inplace=True)
```

Feature Engineering

```
In [40]: df["Date"] = pd.to_datetime(df["Date"])
```

```
In [41]: df["Month"] = df["Date"].dt.month
```

```
In [42]: df["Year"] = df["Date"].dt.year
```

```
In [43]: df["Week"] = df["Date"].dt.isocalendar().week
```

```
In [44]: print(df[["Date", "Month", "Year", "Week"]].head())
```

	Date	Month	Year	Week
0	2010-02-05	2	2010	5
1	2010-02-12	2	2010	6
2	2010-02-19	2	2010	7
3	2010-02-26	2	2010	8
4	2010-03-05	3	2010	9

Convert Categorical Data

```
In [45]: df = pd.get_dummies(df, columns=["Type"], drop_first=True)
```

Select Features (X) and Target (Y)

Target Variable

```
In [46]: y = df["Weekly_Sales"]
```

Input Features

```
In [47]: X = df[[
    "Store",
    "Dept",
    "Temperature",
    "Fuel_Price",
    "CPI",
    "Unemployment",
    "IsHoliday_x",
    "Size",
    "Month",
    "Year",
```

"Week"

```
]]
```

Train-Test Split

```
In [48]: from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

Import ML Libraries

```
In [49]: from xgboost import XGBRegressor

from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)

import numpy as np
```

Create XGBoost Model

```
In [50]: model = XGBRegressor(
    n_estimators=100,
    learning_rate=0.1,
    max_depth=6,
    random_state=42
)
```

Train the Model

```
In [51]: model.fit(X_train, y_train)
```

Out[51]:

```
XGBRegressor
XGBRegressor(base_score=None, booster=None, callbacks=None,
              colsample_bylevel=None, colsample_bynode=None,
              colsample_bytree=None, device=None, early_stopping_round
s=None,
              enable_categorical=False, eval_metric=None, feature_type
s=None,
              feature_weights=None, gamma=None, grow_policy=None,
              importance_type=None, interaction_constraints=None,
              learning_rate=0.1, max_bin=None, max_cat_threshold=None,
              max cat to onehot=None, max delta step=None, max dept
```

In [52]: `y_pred = model.predict(X_test)`

In [53]: `print(y_pred[:10])`

```
[58673.45    2782.631   9633.491   3864.8508  3770.178   8169.735
 4047.5486 11818.368   8009.627  13381.705 ]
```

Evaluate Model Accuracy

Mean Absolute Error (MAE)

In [54]: `mae = mean_absolute_error(y_test, y_pred)`

```
print("MAE:", mae)
```

MAE: 3916.8807673388073

Root Mean Squared Error (RMSE)

In [55]: `rmse = np.sqrt(mean_squared_error(y_test, y_pred))`

```
print("RMSE:", rmse)
```

RMSE: 6856.544710029819

R² Score

In [56]: `r2 = r2_score(y_test, y_pred)`

```
print("R2 Score:", r2)
```

R2 Score: 0.9098469406683599

Compare Actual vs Predicted

```
In [57]: results = pd.DataFrame({
          "Actual": y_test.values,
          "Predicted": y_pred
        })

print(results.head(10))
```

	Actual	Predicted
0	50932.42	58673.449219
1	3196.12	2782.631104
2	10125.03	9633.491211
3	3311.26	3864.850830
4	6335.65	3770.177979
5	8971.23	8169.734863
6	1575.35	4047.548584
7	17308.45	11818.368164
8	5763.39	8009.626953
9	17034.57	13381.705078

Visualization

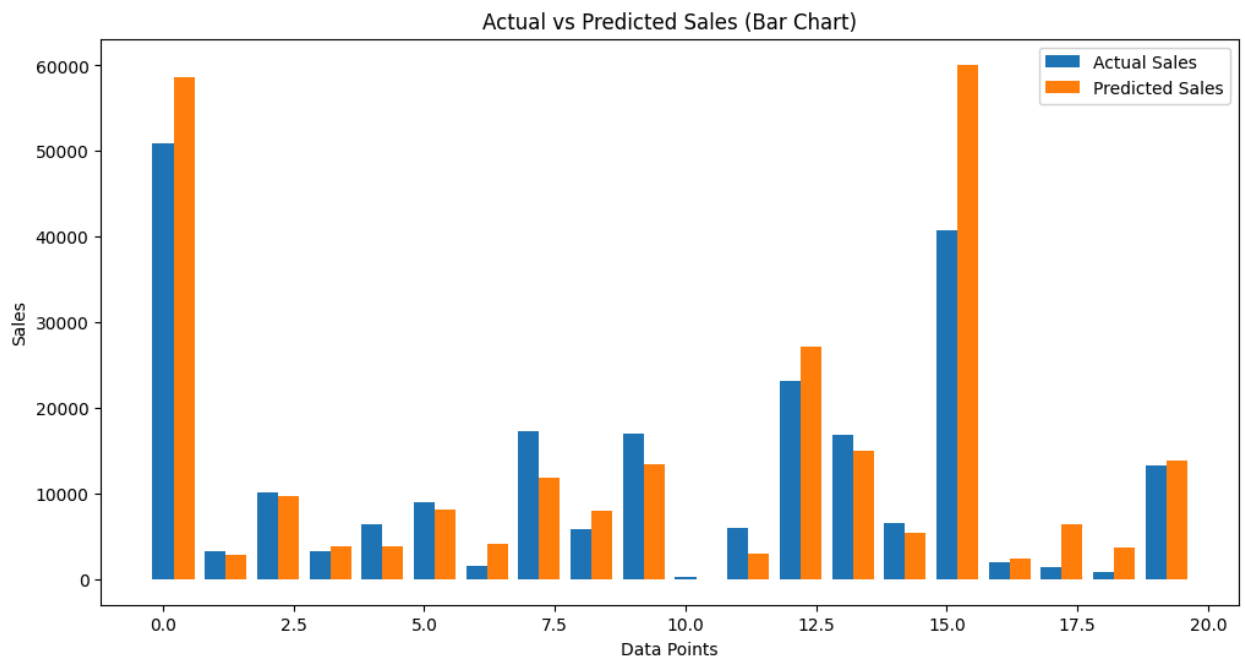
```
In [58]: plt.figure(figsize=(12,6))

x = range(20)

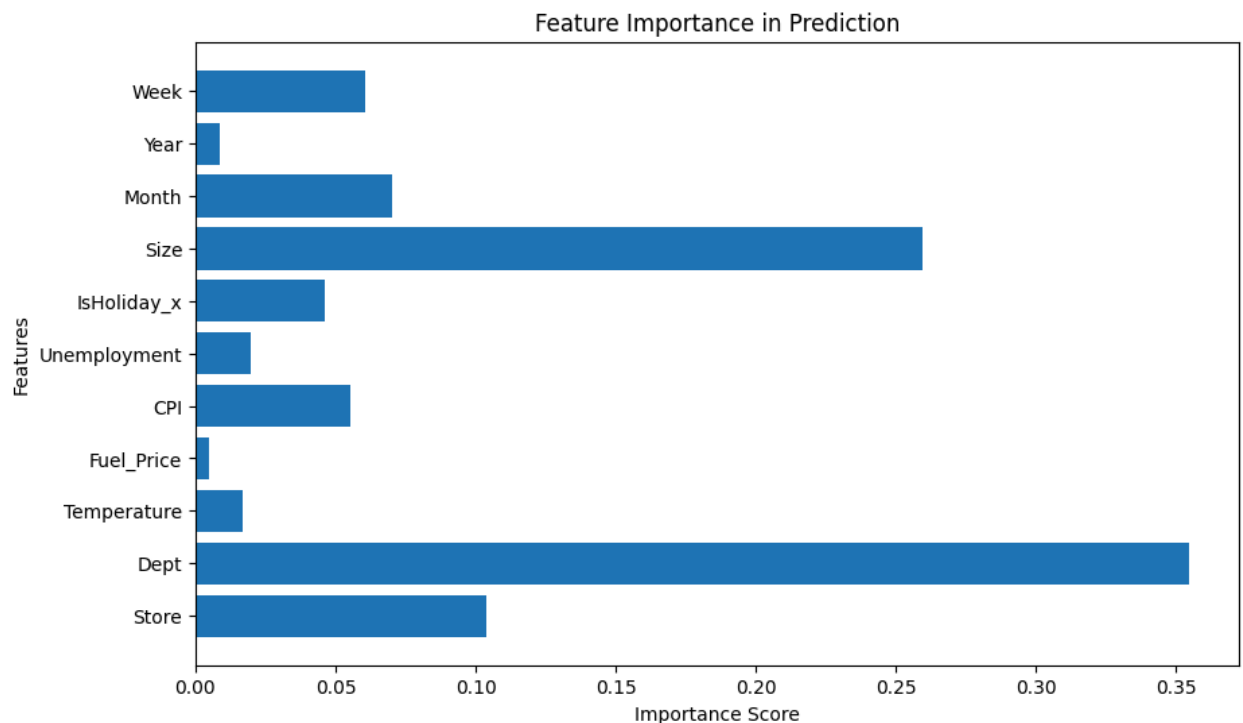
plt.bar(x, y_test.values[:20], width=0.4, label="Actual Sales")
plt.bar([i + 0.4 for i in x], y_pred[:20], width=0.4, label="Predicted Sales")

plt.xlabel("Data Points")
plt.ylabel("Sales")
plt.title("Actual vs Predicted Sales (Bar Chart)")

plt.legend()
plt.show()
```

```
In [59]: importance = model.feature_importances_  
  
features = X.columns  
  
plt.figure(figsize=(10,6))  
  
plt.barh(features, importance)  
  
plt.xlabel("Importance Score")  
plt.ylabel("Features")  
  
plt.title("Feature Importance in Prediction")  
  
plt.show()
```



Create AI Business Recommendation System

```
In [60]: def inventory_recommendation(predicted_sales):  
    if predicted_sales > 50000:  
        return "🔥 High Demand → Increase Inventory"  
    elif predicted_sales > 20000:  
        return "📦 Moderate Demand → Maintain Inventory"  
    else:  
        return "⚠️ Low Demand → Reduce Inventory"
```

Test AI Recommendation

```
In [61]: sample_prediction = y_pred[0]  
print("Predicted Sales:", sample_prediction)  
print(  
    inventory_recommendation(sample_prediction)  
)
```

Predicted Sales: 58673.45

🔥 High Demand → Increase Inventory

Smart Sales Alert System

```
In [62]: def sales_alert(predicted_sales):  
    if predicted_sales > 80000:  
        return "🔥 Very High Sales Spike Detected!"  
    elif predicted_sales < 10000:  
        return "⚠️ Very Low Sales Warning!"  
    else:  
        return "✅ Sales Look Normal"
```

Test It

```
In [63]: print(sales_alert(sample_prediction))
```

✅ Sales Look Normal

```
In [ ]:
```

Final AI Insight Table

```
In [65]: results = pd.DataFrame({  
    "Actual Sales": y_test.values[:10],  
    "Predicted Sales": y_pred[:10]  
})  
  
results["Difference"] = (  
    results["Actual Sales"] - results["Predicted Sales"]  
)  
  
def inventory_recommendation(predicted_sales):  
    if predicted_sales > 50000:  
        return "Increase Inventory"  
    elif predicted_sales > 20000:  
        return "Maintain Inventory"  
    else:  
        return "Reduce Inventory"  
  
results["AI Recommendation"] = results[  
    "Predicted Sales"  
].apply(inventory_recommendation)
```

```
print(results)
```

	Actual Sales	Predicted Sales	Difference	AI Recommendation
0	50932.42	58673.449219	-7741.029219	Increase Inventory
1	3196.12	2782.631104	413.488896	Reduce Inventory
2	10125.03	9633.491211	491.538789	Reduce Inventory
3	3311.26	3864.850830	-553.590830	Reduce Inventory
4	6335.65	3770.177979	2565.472021	Reduce Inventory
5	8971.23	8169.734863	801.495137	Reduce Inventory
6	1575.35	4047.548584	-2472.198584	Reduce Inventory
7	17308.45	11818.368164	5490.081836	Reduce Inventory
8	5763.39	8009.626953	-2246.236953	Reduce Inventory
9	17034.57	13381.705078	3652.864922	Reduce Inventory

AI Insight Bar Visualization

```
In [66]: plt.figure(figsize=(12,6))

results[["Actual Sales", "Predicted Sales"]].plot(
    kind="bar",
    figsize=(12,6)
)

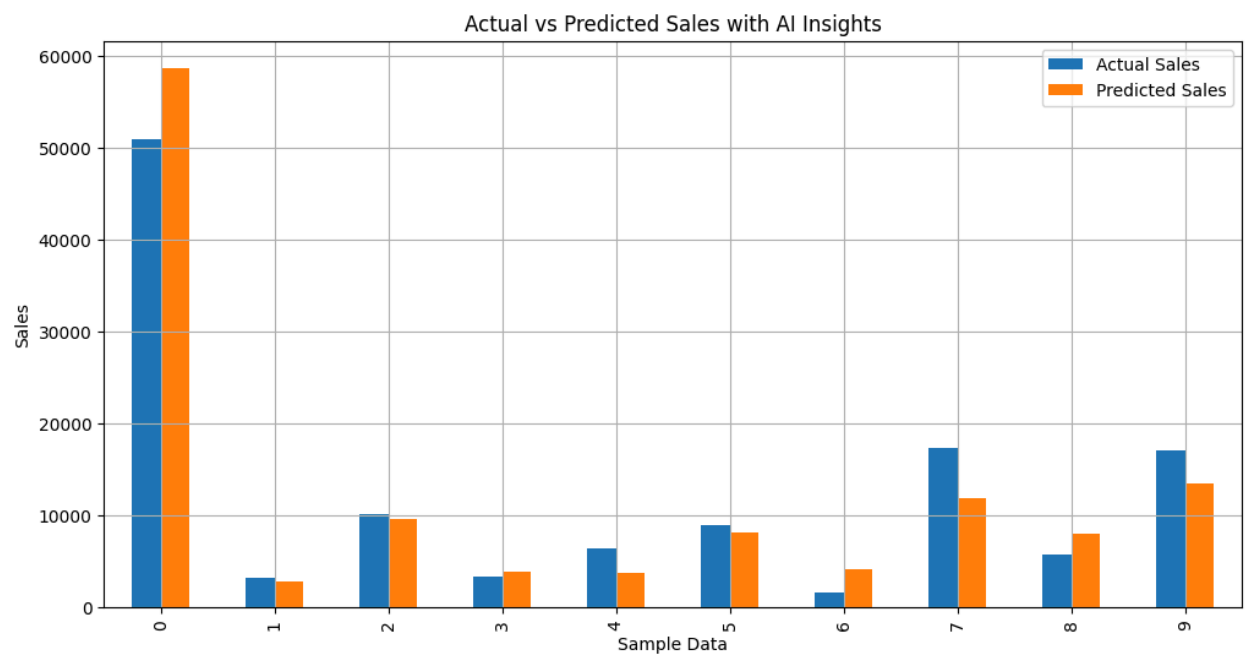
plt.title("Actual vs Predicted Sales with AI Insights")

plt.xlabel("Sample Data")
plt.ylabel("Sales")

plt.grid(True)

plt.show()
```

<Figure size 1200x600 with 0 Axes>



Get OpenAI Library

```
In [67]: !pip install openai
```

```
Requirement already satisfied: openai in /usr/local/lib/python3.12/dist-packages (2.37.0)
Requirement already satisfied: anyio<5,>=3.5.0 in /usr/local/lib/python3.12/dist-packages (from openai) (4.13.0)
Requirement already satisfied: distro<2,>=1.7.0 in /usr/local/lib/python3.12/dist-packages (from openai) (1.9.0)
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from openai) (0.28.1)
Requirement already satisfied: jiter<1,>=0.10.0 in /usr/local/lib/python3.12/dist-packages (from openai) (0.15.0)
Requirement already satisfied: pydantic<3,>=1.9.0 in /usr/local/lib/python3.12/dist-packages (from openai) (2.12.3)
Requirement already satisfied: sniffio in /usr/local/lib/python3.12/dist-packages (from openai) (1.3.1)
Requirement already satisfied: tqdm>4 in /usr/local/lib/python3.12/dist-packages (from openai) (4.67.3)
Requirement already satisfied: typing-extensions<5,>=4.11 in /usr/local/lib/python3.12/dist-packages (from openai) (4.15.0)
Requirement already satisfied: idna>=2.8 in /usr/local/lib/python3.12/dist-packages (from anyio<5,>=3.5.0->openai) (3.15)
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->openai) (2026.5.20)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->openai) (1.0.9)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<1,>=0.23.0->openai) (0.16.0)
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.12/dist-packages (from pydantic<3,>=1.9.0->openai) (0.7.0)
Requirement already satisfied: pydantic-core==2.41.4 in /usr/local/lib/python3.12/dist-packages (from pydantic<3,>=1.9.0->openai) (2.41.4)
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.12/dist-packages (from pydantic<3,>=1.9.0->openai) (0.4.2)
```

```
In [68]: import pandas as pd
```

Create Simple AI Chatbot

```
In [84]: def walmart_ai_chatbot(question):

    question = question.lower()

    # -----
    # SALES DROP
    # -----

    if "sales dropped" in question:
```

```

        return """
Possible reasons for sales drop:
- Non-holiday week
- Lower customer demand
- Seasonal effect
- Fuel price increase
- Economic slowdown
"""

# -----
# SALES INCREASE
# -----

elif "sales increased" in question:

    return """
Possible reasons for sales increase:
- Holiday season
- Discounts and offers
- Increased customer demand
- Festival shopping
- Seasonal products
"""

# -----
# HIGHEST SALES
# -----

elif "highest sales" in question:

    max_sales = df["Weekly_Sales"].max()

    return f"""
Highest weekly sales recorded:
{max_sales}
"""

# -----
# AVERAGE SALES
# -----

elif "average sales" in question:

    avg_sales = df["Weekly_Sales"].mean()

    return f"""
Average weekly sales:
{avg_sales}
"""

# -----
# SALES TREND

```

```

# -----

elif "sales trend" in question:

    return """
Sales trend shows:
- Seasonal fluctuations
- Higher holiday sales
- Weekend demand spikes
- Store performance variations
"""

# -----
# PEAK SALES
# -----

elif "peak" in question:

    return """
Sales usually peak during:
- Christmas
- Black Friday
- Thanksgiving
- Holiday weekends
"""

# -----
# BEST STORE
# -----

elif "best store" in question:

    best_store = df.groupby(
        "Store"
    )["Weekly_Sales"].mean().idxmax()

    return f"""
Store {best_store} has the highest average sales.
"""

# -----
# LOWEST SALES STORE
# -----

elif "lowest sales" in question:

    low_store = df.groupby(
        "Store"
    )["Weekly_Sales"].mean().idxmin()

    return f"""
Store {low_store} has the lowest average sales.
"""

```

```

# -----
# TOP 5 STORES
# -----

elif "top 5 stores" in question:

    top_stores = df.groupby(
        "Store"
    )["Weekly_Sales"].mean().sort_values(
        ascending=False
    ).head()

    return f"""
Top 5 stores by average sales:

{top_stores}
"""

# -----
# STORE IMPROVEMENT
# -----

elif "improvement" in question:

    return """
Stores with lower sales may need:
- Better marketing
- Seasonal promotions
- Inventory optimization
- Customer engagement strategies
"""

# -----
# PROFITABLE STORES
# -----

elif "profitable stores" in question:

    return """
Highly profitable stores usually:
- Have consistent demand
- Perform well during holidays
- Maintain inventory efficiently
"""

# -----
# STOCK MORE
# -----

elif "stock more" in question:

    return """

```


Recommended products to stock more:

- Grocery items
- Holiday products
- Electronics
- High-demand seasonal products

"""

```
# -----  
# HIGH DEMAND PRODUCTS  
# -----
```

```
elif "high demand" in question:
```

```
    return """
```

High demand products include:

- Grocery items
- Winter products
- Electronics
- Holiday gifts

"""

```
# -----  
# INVENTORY INCREASE  
# -----
```

```
elif "increase inventory" in question:
```

```
    return """
```

Inventory should increase during:

- Holiday season
- Weekend sales spikes
- Promotional campaigns

"""

```
# -----  
# HOLIDAY PRODUCTS  
# -----
```

```
elif "holiday products" in question:
```

```
    return """
```

Popular holiday products:

- Decorations
- Electronics
- Toys
- Gift items

"""

```
# -----  
# LOW DEMAND CATEGORY  
# -----
```

```
elif "low demand" in question:
```

```

        return """
Low demand products may require:
- Discounts
- Better marketing
- Reduced inventory
"""

# -----
# HOLIDAY SALES
# -----

elif "holidays increase sales" in question:

    return """
Yes, holidays significantly increase Walmart sales
due to higher customer shopping activity.
"""

# -----
# BEST HOLIDAY
# -----

elif "highest holiday sales" in question:

    return """
Black Friday and Christmas usually generate
the highest sales.
"""

# -----
# HOLIDAY VS NON-HOLIDAY
# -----

elif "holiday vs non-holiday" in question:

    return """
Holiday weeks generally perform much better
than non-holiday weeks.
"""

# -----
# TEMPERATURE EFFECT
# -----

elif "temperature affect sales" in question:

    return """
Yes, temperature affects seasonal product demand
such as winter and summer products.
"""

# -----

```

```

# FUEL PRICE EFFECT
# -----

elif "fuel price" in question:

    return """
Higher fuel prices may reduce customer visits
and impact sales negatively.
"""

# -----
# UNEMPLOYMENT EFFECT
# -----

elif "unemployment" in question:

    return """
Higher unemployment can reduce customer spending
and lower Walmart sales.
"""

# -----
# MOST IMPORTANT FACTOR
# -----

elif "impacts sales most" in question:

    return """
Important factors affecting sales:
- Holidays
- Store size
- Seasonal demand
- Fuel price
- Temperature
"""

# -----
# BUSINESS RECOMMENDATIONS
# -----

elif "business recommendations" in question:

    return """
AI Recommendations:
- Increase holiday inventory
- Focus on top-performing stores
- Improve low-performing stores
- Use seasonal promotions
"""

# -----
# IMPROVE SALES
# -----

```

```

elif "improve sales" in question:

    return """
Ways to improve sales:
- Better marketing
- Holiday offers
- Product recommendations
- Personalized discounts
"""

# -----
# MARKETING STORES
# -----

elif "need marketing" in question:

    return """
Stores with lower performance may benefit from:
- Local advertising
- Discount campaigns
- Loyalty programs
"""

# -----
# INVESTMENT
# -----

elif "invest more" in question:

    return """
Walmart should invest more in:
- High-performing stores
- Online sales
- Holiday inventory
- AI forecasting systems
"""

# -----
# FUTURE RISKS
# -----

elif "future sales risks" in question:

    return """
Potential risks:
- Economic slowdown
- Supply chain issues
- Inflation
- Seasonal demand changes
"""

# -----

```

```

# FUTURE TREND
# -----

elif "future sales trend" in question:

    return """
Future sales may increase during:
- Holiday seasons
- Promotional events
- Weekend demand spikes
"""

# -----
# SALES GROWTH
# -----

elif "sales grow" in question:

    return """
Sales are likely to grow during
high-demand shopping seasons.
"""

# -----
# SALES DECREASE
# -----

elif "sales decrease" in question:

    return """
Sales may decrease during:
- Non-holiday periods
- Economic downturns
- Low customer demand
"""

# -----
# EXPLAIN PREDICTION
# -----

elif "explain this prediction" in question:

    return """
Prediction is influenced by:
- Holiday effect
- Store performance
- Fuel price
- Temperature
- Seasonal trends
"""

# -----
# HIGH SALES PREDICTION

```

```

# -----

elif "high sales" in question:

    return """
AI predicted high sales because:
- Strong seasonal demand
- Holiday shopping
- High-performing stores
"""

# -----
# IMPORTANT FEATURES
# -----

elif "important sales factors" in question:

    return """
Most important sales factors:
- Holidays
- Store size
- Fuel price
- Temperature
- Unemployment
"""

# -----
# DEFAULT RESPONSE
# -----

else:

    return

```

Ask AI Questions

```

In [70]: question = input("Ask AI Assistant: ")

response = walmart_ai_chatbot(question)

print(response)

```

Ask AI Assistant: Why sales dropped?

Possible reasons for sales drop:

- Non-holiday week
- Lower customer demand
- Seasonal effect
- Fuel price increase
- Economic slowdown

```
In [72]: question = input("Ask AI Assistant: ")
        response = walmart_ai_chatbot(question)
        print(response)
```

Ask AI Assistant: Which best store?

Store 20 has the highest average sales.

```
In [73]: question = input("Ask AI Assistant: ")
        response = walmart_ai_chatbot(question)
        print(response)
```

Ask AI Assistant: What should Walmart stock more?

Recommended products to stock more:

- Grocery items
- Holiday products
- Electronics
- High-demand seasonal products

```
In [74]: question = input("Ask AI Assistant: ")
        response = walmart_ai_chatbot(question)
        print(response)
```

Ask AI Assistant: highest sales

Highest weekly sales recorded:
693099.36

```
In [75]: question = input("Ask AI Assistant: ")
        response = walmart_ai_chatbot(question)
        print(response)
```

Ask AI Assistant: How does unemployment affect Walmart sales?

I can answer:

- Why sales dropped
- Best performing store
- Stock recommendations
- Highest sales analysis

```
In [85]: question = input("Ask AI Assistant: ")
```

```
response = walmart_ai_chatbot(question)

print(response)
```

Ask AI Assistant: Explain this prediction

Prediction is influenced by:

- Holiday effect
- Store performance
- Fuel price
- Temperature
- Seasonal trends

ADD INVENTORY OPTIMIZATION AI

```
In [94]: predicted_sales = model.predict(X_test)
```

```
In [95]: # Business Inputs
lead_time_weeks = 2          # time to restock
service_level = 1.65         # 95% confidence level
```

Create file and full pipeline function

```
In [101]: %%writefile inventory_optimization.py

import numpy as np

def calculate_safety_stock(std_dev, lead_time, z):
    return z * std_dev * np.sqrt(lead_time)

def get_optimal_stock(predicted_sales, safety_stock):
    return predicted_sales + safety_stock

def inventory_optimizer(predicted_sales, actual_sales):
    std_dev = np.std(actual_sales)

    lead_time = 2
    z = 1.65

    safety_stock = calculate_safety_stock(std_dev, lead_time, z)
    optimal_stock = get_optimal_stock(predicted_sales, safety_stock)

    return {
        "predicted_sales": predicted_sales,
        "safety_stock": safety_stock,
        "optimal_stock": optimal_stock
    }
```

Writing inventory_optimization.py


```
In [102... from inventory_optimization import inventory_optimizer
```

CONNECT WITH MAIN MODEL

```
In [103... from inventory_optimization import inventory_optimizer
```

```
result = inventory_optimizer(predicted_sales, df['Weekly_Sales'])  
print(result)
```

```
{'predicted_sales': array([58673.45 , 2782.631, 9633.491, ..., 12031.019, 144  
15.285,  
28089.977], dtype=float32), 'safety_stock': np.float64(52995.402333149  
7), 'optimal_stock': array([111668.8515519 , 55778.03343667, 62628.89354409,  
...,  
65026.42088784, 67410.6874894 , 81085.37889565])}
```

ADD OVERSTOCK / UNDERSTOCK DETECTION

```
In [104... def stock_risk(current_stock, optimal_stock):
```

```
    if current_stock > optimal_stock:  
        return "⚠ Overstock Risk"  
  
    elif current_stock < optimal_stock:  
        return "🔥 Understock Risk"  
  
    else:  
        return "✅ Balanced Stock"
```

ADD TO DASHBOARD

```
In [109... print("Inventory Optimization AI")  
  
print("Predicted Sales:", predicted_sales)  
print("Safety Stock:", result['safety_stock'])  
print("Optimal Stock:", result['optimal_stock'])
```

```
Inventory Optimization AI  
Predicted Sales: [58673.45 2782.631 9633.491 ... 12031.019 14415.285 28089.9  
77]  
Safety Stock: 52995.4023331497  
Optimal Stock: [111668.8515519 55778.03343667 62628.89354409 ... 65026.4208  
8784  
67410.6874894 81085.37889565]
```

```
In [110... import matplotlib.pyplot as plt

labels = ["Predicted", "Safety Stock", "Optimal", "Current"]
values = [12000, 2000, 14000, 10000]

plt.bar(labels, values)
plt.title("Inventory Optimization Overview")
plt.show()
```

